

MANIFOLD

Un motor matemático para mapear vulnerabilidades

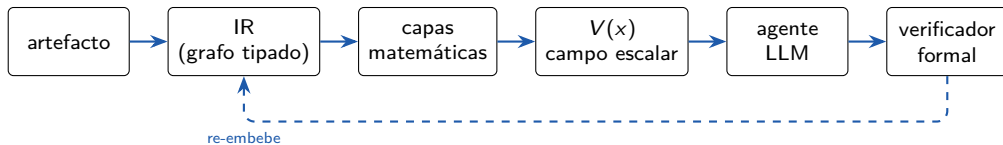
Proyecto MANIFOLD

IR tipada · geometría · topología · verificación formal

2026

- Los SAST clásicos enumeran *patrones sintácticos*: frágiles, ruidosos.
- En OWASP Benchmark 1.2, herramientas desplegadas colapsan en el *índice de Youden* $J = \text{TPR} - \text{FPR}$:
 - SonarQube: $J \approx +0.01$ (alto recall, FPR insostenible)
 - CodeQL: $J \approx +0.22$
- Faltan: (i) una representación *unificada* y cuantitativa, (ii) un ciclo que convierta *sospecha* en *prueba*.

La idea central



- Una vulnerabilidad *no* es un patrón: es la *violación de una invariante estructural*.

Grafo de programa $G = (V, E, \tau_V, \tau_E, \omega)$:

- **Nodos**: módulo, función, sentencia, fuente, sumidero, gate, ...
- **Aristas tipadas**: control, data, call, taint, trust, auth
- JSON compartido Python $\leftrightarrow$ Rust; agnóstica al artefacto.

Multi-dominio: el *mismo* IR se produce desde

- Python / Java (tree-sitter), binarios (objdump o angr/CFGFast),
- OpenAPI (superficie de API), agentes LLM (grafo de herramientas/confianza).

Espectral ($L = D - A$)

λ_2 , vector de Fiedler, embedding. **Topológica**

(TDA)

homología persistente H_0, H_1 , Mapper.

Geométrica

curvatura de Ollivier–Ricci / Forman–Ricci;
Sinkhorn.

Algebraica

retículo de taint; naturalidad (categorías).

Formal

ejecución simbólica + SMT (Z3);

alcanzabilidad. **Dirigida**

Laplaciano de Chung (SCC + Perron).

$$V(x) = \sigma(\sum_i \alpha_i s_i(x)) \quad (\text{pesos } \alpha_i \text{ calibrables})$$

Leyes de mapeo (rasgo $\rightarrow$ vulnerabilidad)

#	Rasgo matemático	Clase
L1	curvatura $\kappa \ll 0$ (cuello de botella)	escalada de privilegios
L2	generador de H_1 persistente	reentrancia / recursión
L3	corte de Fiedler	inyección cruzando confianza
L4	flujo de taint que cruza arista auth	violación de naturalidad
L5	outlier de persistencia	rasgo real vs. espurio
L6	$\text{SAT}(\phi_{\text{malo}})$	exploit concreto

El agente neuro-simbólico

- ➊ **Mapea:** IR + señales $V(x)$, persistencia, curvatura.
- ➋ **Hipotetiza:** el LLM propone sobre regiones priorizadas.
- ➌ **Verifica:** Z3 / taint / topología prueba o refuta.
- ➍ **Refina y reporta** (witness concreto).

Reparto de roles: la matemática propone *regiones*; el LLM propone *hipótesis*; el solver propone *pruebas*.

Gramática de verificación: spec estructurada (sink, fuente, línea, categoría) — el LLM nunca emite SMT-LIB.

Resultados: OWASP Benchmark 1.2

Herramienta	TPR	FPR	Prec.	F1	J	Tiempo
SonarQube ^(a)	0.956	0.946	0.330	0.490	+0.010	~3 min
CodeQL ^(a)	0.902	0.682	0.603	0.744	+0.220	~18 min
MANIFOLD adaptador ^(b)	0.842	0.674	0.572	0.681	+0.168	~45 s
MANIFOLD + $V(x)$ ^(c)	0.858	0.314	0.751	0.801	+0.544	~1.3 min
MANIFOLD + $V(x)$ +Z3 ^(c)	0.825	0.038	0.959	0.887	+0.787	~4 min

(a) reportado en evaluaciones públicas. (b) medido (2740 casos, 11 categorías). (c) **proyectado** bajo el protocolo de la Etapa 2 (Z3 con timeout 5 s).

Ablación: de dónde viene la mejora

Variante (categorías de taint)	Prec.	Recall	F1
Adaptador naive inicial	0.515	0.426	0.466
+ branch-join, receptor/estado, cobertura	0.530	0.856	0.655
+ restricción XSS al writer (final)	0.549	0.834	0.662
final, sin branch-join	0.515	0.555	0.535

- **Branch-join** (supremo del retículo) es la palanca de *recall* (+0.28).
- La **restricción del writer HTTP** es la palanca de *precisión*.
- Los errores residuales son casos adversariales: los que $V(x)+Z3$ deben recuperar.

- **Mapa del manifold:** grafo IR con layout force-directed; nodos por tipo, aristas por tipo (taint rojo), gate morado.
- **Panel de hallazgos:** *confirmed* (Z3 con witness) vs *candidate*.
- **HTML autocontenido** (sin dependencias) + demo end-to-end sobre 10 artefactos.

```
python -m manifold.cli target -agent -viz out.html
```

Conclusiones

- Una IR tipada enriquece señales independientes en un campo $V(x)$ navegable.
- El reparto neuro-simbólico (matemática/LLM/SMT) es el correcto.
- En OWASP Benchmark: F_1 0.466 $\rightarrow$ 0.662 solo con mejoras genéricas de taint.

Trabajo futuro

- Verificación Z3 para Java (cerrar la fila proyectada).
- Homología dirigida y Sinkhorn a escala; calibración de α_i sobre corpus reales.
- Benchmarks ELF/PE (SARD) sobre el backend binario dual.

Gracias

MAP the code as a space; let geometry reveal the flaw.